Module 3

Python OOP Concepts

1. Object-Oriented Programming (OOP)

OOP is a programming approach that organizes code using classes and objects. It helps make programs reusable, organized, and easier to maintain.

Example:

```
class Student:
    def display(self):
        print("Hello Student")

s1 = Student()
s1.display()
```

Output

```
Hello Student
```

2. Classes and Objects

Class: A blueprint used to create objects.

Object: An instance of a class that can have data and methods.

Example:

```
class Car:
    def start(self):
        print("Car started")

car1 = Car()
car1.start()
```

Output

```
Car started
```

3. Constructors

A constructor initializes an object's data when the object is created. In Python, `__init__()` is commonly used for this purpose.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name

s1 = Student("Vardhini")
print(s1.name)
```

Output

Vardhini

4. Methods

Methods are functions defined inside a class. They describe the actions an object can perform.

Example:

```
class Calculator:
    def add(self, a, b):
        return a + b

c = Calculator()
print(c.add(10, 20))
```

Output

30

5. Encapsulation

Encapsulation means combining data and methods in a class and controlling how the data is accessed or changed.

In Python, a double underscore such as `__balance` helps prevent direct access through the usual attribute name.

Example:

```
class Account:
    def __init__(self, balance):
        self.__balance = balance

    def show_balance(self):
        print(self.__balance)

account = Account(1000)
account.show_balance()
```

Output

1000

6. Inheritance

Inheritance allows one class to reuse and extend the properties and methods of another class.

- Parent class: the class being inherited from.
- Child class: the class that inherits.

Example:

```
class Account:
    def display(self):
        print("Bank Account")

class SavingsAccount(Account):
    pass

a = SavingsAccount()
a.display()
```

Output

Bank Account

7. Polymorphism

Polymorphism means “many forms.” Different classes can provide their own implementations of a method with the same name.

Example:

```
class Dog:
    def sound(self):
        print("Bark")

class Cat:
    def sound(self):
        print("Meow")

for animal in [Dog(), Cat()]:
    animal.sound()
```

Output

```
Bark
Meow
```

8. Abstraction

Abstraction means hiding implementation details and showing only the essential operations. Python supports abstraction through the `abc` module and abstract methods.

Example:

```
from abc import ABC, abstractmethod

class Account(ABC):
    @abstractmethod
    def account_type(self):
        pass

class SavingsAccount(Account):
```

```
    def account_type(self):
        return "Savings Account"

a = SavingsAccount()
print(a.account_type())
```

Output:

Savings Account

9. Access Modifiers and Private Attributes

Access conventions indicate how class attributes and methods should be used.

- `name` — public; can be accessed normally.
- `_name` — protected by convention; intended for internal or subclass use.
- `__name` — name-mangled; helps avoid accidental direct access.

Example:

```
class Student:
    def __init__(self):
        self.name = "Vardhini"
        self._course = "AIML"
        self.__marks = 90

s = Student()
print(s.name)
print(s._course)
```

Output:

Vardhini
AIML

10. Exception Handling

Exception handling manages runtime errors so the program can respond appropriately instead of terminating unexpectedly.

The main keywords are `try`, `except`, and `finally`.

Example:

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)

except ValueError:
    print("Enter a valid number.")

except ZeroDivisionError:
    print("Cannot divide by zero.")

finally:
    print("Program completed.")
```

11. Input Validation

Input validation checks whether the entered data is acceptable before processing it.

For example, a bank should reject a deposit amount that is zero or negative.

Example:

```
amount = float(input("Enter deposit amount: "))

if amount > 0:
    print("Valid deposit")
else:
    print("Amount must be positive")
```

12. Python Modules and Imports

A module is a Python file containing reusable code. The `import` statement allows one file to use code from another file or library.

Example:

```
# main.py
from calculator import add

print(add(5, 3))
```

Output:

8

13. Lists and Dictionaries

- **List: An ordered collection of items.**
- **Dictionary: Stores information as key-value pairs.**

Example:

```
students = ["Ravi", "Vardhini", "Anu"]

print(students[0])



account = {

    "account_number": "101",

    "owner": "Vardhini",

    "balance": 5000

}

print(account["balance"])
```

Output:

Ravi

5000

14. Banking Operations

Banking operations are actions performed on an account, such as creating an account, depositing money, withdrawing money, and checking the balance.

Example:

```
balance = 1000


balance += 500

print("After deposit:", balance)


balance -= 200

print("After withdrawal:", balance)
```

Output:

After deposit: 1500

After withdrawal: 1300

15. Account Creation

Account creation means creating a new account object with details such as account number, account holder name, and initial balance.

Example:

```
class Account:

    def __init__(self, number, owner, balance=0):
```

```
        self.number = number
        self.owner = owner
        self.balance = balance

account1 = Account("101", "Vardhini", 1000)

print(account1.owner)
print(account1.balance)
```

16. Deposit and Withdrawal

Deposit: Adds money to an account.

Withdrawal: Removes money from an account if sufficient funds are available.

Example:

```
class Account:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount

    def withdraw(self, amount):
```

```
        if amount <= 0:
            print("Enter a positive amount.")
        elif amount > self.balance:
            print("Insufficient balance.")
        else:
            self.balance -= amount

account = Account(1000)
account.deposit(500)
account.withdraw(200)

print(account.balance)
```

Output:

1300

17. Transaction History

Transaction history is a record of past deposits and withdrawals, including the transaction type, amount, date, and resulting balance.

Example:

```
transactions = []

transactions.append("Deposited ₹500")
transactions.append("Withdrawn ₹200")
```

```
for transaction in transactions:
    print(transaction)
```

Output:

Deposited ₹500

Withdrawn ₹200

18. Clean Code Principles

Clean code is easy to read, understand, test, and maintain.

Important principles:

- Use meaningful variable and function names.
- Keep functions focused on one task.
- Avoid unnecessary repeated code.
- Use consistent indentation.
- Add comments when they clarify important logic.
- Handle errors appropriately.
- Separate the program into modules when useful.

Example:

```
deposit_amount = 500

current_balance = 200

updated_balance = current_balance + deposit_amount
```

19. Class Diagram and Application Architecture

A class diagram is a visual representation of classes, their attributes, methods, and relationships.

For your Banking System:

Account (Abstract)

account_number, owner, balance

deposit() · withdraw() · show_history()

↑ Inheritance

SavingsAccount

account_type()

Account stores transaction records

Transaction

transaction_type, amount, date, balance

display()

A clean architecture separates responsibilities:

- `main.py` — user menu and program flow.
- `account.py` — account rules and banking operations.
- `savings_account.py` — savings account implementation.
- `transaction.py` — transaction details and display.
- `class_diagram.md` — architecture documentation.